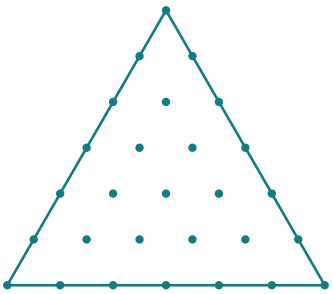
PROBABILITY AND MATHEMATICAL MODELLING

Discrete Information Geometry

Probability, Counting,
and Finite Dynamics

Chapter 5



A course in ten chapters

https://bobovski66.github.io/IntroDiscreteInfoGeom/

CHAPTER 5

Entropy, code trees, and multiplicity

A message that was very likely conveys less surprise than a message that was very unlikely. Logarithms turn this idea into an additive numerical measure. The same measure appears in binary code lengths and in the number of ordered samples with a specified composition. These connections give entropy a finite combinatorial interpretation.

5.1 Logarithms measure multiplicative scales

For $a > 0$, $\log_2 a$ is the exponent to which 2 must be raised to obtain a. Thus $\log_2 8 = 3$ and $\log_2(1/4) = -2$. The natural logarithm $\ln a$ uses base e. The conversion formula is $\log_2 a = \ln a / \ln 2$.

The identities

$$\ln(ab) = \ln a + \ln b, \qquad \ln(a^r) = r \ln a$$

allow us to express repeated multiplication as addition. The logarithm is increasing, so it preserves inequalities between positive numbers. No method for differentiating logarithms is needed for these uses.

The **surprisal** of an outcome of probability $p > 0$ is $-\log_2 p$ bits. An event of probability $1/2$ has one bit of surprisal; an event of probability $1/8$ has three bits. For independent outcomes with probabilities p and q, their combined surprisal is the sum because $-\log_2(pq) = -\log_2 p - \log_2 q$.

5.2 Entropy is average surprisal

For a finite distribution p, its **Shannon entropy in bits** is

$$H_2(p) = -\sum_i p_i \log_2 p_i.$$

Using natural logarithms gives entropy in **nats**:

$$H(p) = -\sum_i p_i \ln p_i = H_2(p) \ln 2.$$

A zero-probability outcome is assigned contribution zero to the entropy sum. This convention reflects that the outcome is never observed; it avoids evaluating its individual infinite surprisal inside an average.

Example 5.1 (Three distributions on the same labels)**.** The distribution $(1, 0, 0, 0)$ has entropy zero bits. The distribution $(1/2, 1/4, 1/4, 0)$ has entropy $1/2 + 1/2 + 1/2 = 3/2$ bits. The uniform distribution on four labels has entropy 2 bits. The label set alone does not determine uncertainty; the probabilities matter.

Entropy is always nonnegative, since $-\ln p_i \geq 0$ for $0 < p_i \leq 1$. It is at most $\ln k$ for a distribution on k labels. For a positive distribution, weighted arithmetic-geometric mean gives

$$\prod_i (1/p_i)^{p_i} \leq \sum_i p_i (1/p_i) = k.$$

Taking logarithms proves $H(p) \leq \ln k$. If some entries vanish, apply the same argument on the support, the set of labels with positive probability. Equality at the $\ln k$ bound requires all k probabilities to equal $1/k$.

5.3 Binary trees turn messages into bit strings

A **prefix code** assigns each label a bit string such that no string is the beginning of another. A binary tree represents the code: a left branch contributes 0, a right branch contributes 1, and a label occupies a leaf. Decoding proceeds along branches until it reaches a leaf, after which it starts again at the root.

Example 5.2 (A code matched to three probabilities)**.** For probabilities $(1/2, 1/4, 1/4)$, assign $A \mapsto 0$, $B \mapsto 10$, $C \mapsto 11$. The codeword lengths are $(1, 2, 2)$, so their expected length is $(1/2)1 + (1/4)2 + (1/4)2 = 3/2$ bits. This equals the entropy because each codeword has length $-\log_2 p_i$.

If three labels are equally probable, the same code has expected length $5/3$ bits, while the entropy is $\log_2 3 \approx 1.5850$ bits. Individual code lengths must be whole numbers, which can prevent a single-symbol code from attaining the entropy exactly. Coding blocks of several symbols permits finer averages.

For a prefix tree with longest code length L, a leaf at depth ℓ_i occupies $2^{L-\ell_i}$ positions in a complete depth-L tree. Different leaves occupy disjoint positions. Consequently,

$$\sum_i 2^{-\ell_i} \leq 1.$$

This is the **Kraft inequality**. It is a counting restriction on possible code lengths. Chapter 6 will use relative entropy to show that every such code has expected length at least $H_2(p)$.

5.4 Multiplicity and entropy

Let counts n_i sum to N, and set $q_i = n_i/N$. The multiplicity $\Omega = N!/\prod_i n_i!$ counts ordered samples with empirical distribution q. Under the distribution q itself, every such ordered sample has probability

$$\prod_i q_i^{n_i} = \exp\left(N \sum_i q_i \ln q_i\right) = e^{-NH(q)}.$$

Therefore the probability of the entire collection of samples with this composition is

$$\Omega e^{-NH(q)} \leq 1.$$

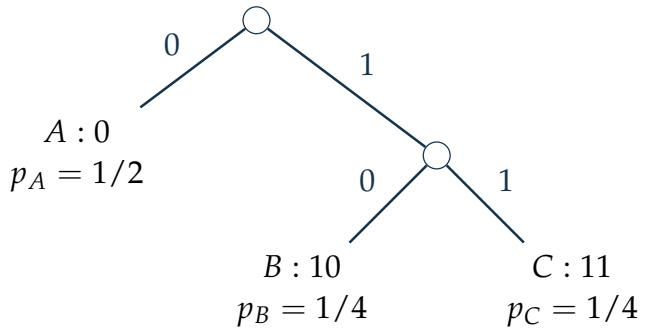


Figure 5.1. A binary prefix tree with code lengths 1, 2, and 2, matched to probabilities (1/2,1/4,1/4).

Taking logarithms gives the exact finite bound

$$\frac{\ln \Omega}{N} \le H(q).$$

The left side is a logarithmic count per observation. It is generally smaller than the entropy at finite N. This inequality requires only counting and the fact that an event cannot have probability greater than one.

Example 5.3 (A balanced sample of size four)**.** For binary counts $(2, 2)$, we have $\Omega = 6$. The logarithmic count per observation is $\log_2 6/4 \approx 0.6462$ bits, while $H_2(1/2, 1/2) = 1$ bit. The probability of exactly two successes under a fair binary model is $6/16 = 3/8$. It is not the whole sample space, which explains the gap in the finite bound.

Many different count vectors are possible, so a single composition need not account for most samples. Entropy captures the exponential scale associated with a composition, while the exact multinomial coefficient records its finite multiplicity. We can compare the two numerically without assuming they are equal.

5.5 Logarithms also simplify dynamics

For exponential selection, the ratio of two positive coordinates satisfies

$$\frac{p_i^+}{p_j^+} = \frac{p_i}{p_j} e^{\eta(f_i - f_j)}.$$

Taking natural logarithms yields the exact **log-odds increment**

$$\ln \frac{p_i^+}{p_j^+} - \ln \frac{p_i}{p_j} = \eta(f_i - f_j).$$

This is an additive difference equation. If scores are constant, repeated addition solves it. More directly,

$$p_{i,t} = \frac{p_{i,0} e^{t\eta f_i}}{\sum_j p_{j,0} e^{t\eta f_j}}.$$

When scores depend on the current state, the log-odds increment still holds for each step, but the score differences must be summed along the trajectory.

Example 5.4 (Accumulating evidence in odds)**.** Suppose two hypotheses have initial odds $1 : 4$. A sequence of observations has likelihood ratios 2, 3, and $1/2$ in favor of the first hypothesis. Multiplying gives final odds $3 : 4$, so its final probability is $3/7$. Equivalently, the log-odds change by $\ln 2 + \ln 3 - \ln 2 = \ln 3$. Likelihood ratios are multiplicative weights, a connection we will revisit in learning updates.

5.6 Entropy of a pair of observations

Let X and Y have joint probabilities $p_{ij} = P(X = i, Y = j)$. Their joint entropy is $H(X, Y) = -\sum_{i,j} p_{ij} \ln p_{ij}$. The marginal probability of $X = i$ is $p_i = \sum_j p_{ij}$. When $p_i > 0$, write $r_{j|i} = p_{ij}/p_i$ for the conditional table of Y given that value of X.

Using $p_{ij} = p_i r_{j|i}$ and splitting the logarithm gives the **chain rule**

$$H(X, Y) = H(X) + \sum_i p_i H(Y \mid X = i).$$

The second term averages the entropy of the conditional tables. Values of i with probability zero contribute zero. The formula states that describing a pair can be organized as describing the first label and then describing the second label given the first.

If the observations are independent, every conditional table equals the marginal table of Y, so $H(X, Y) = H(X) + H(Y)$. Applying this repeatedly gives entropy $NH(p)$ for an independent length-N sample from p. This is the average surprisal of an ordered sample; it is different from the logarithm of the number of samples in one count class.

Example 5.5 (Dependence changes joint uncertainty)**.** Let X be a fair binary label. If $Y = X$, the two possible pairs are $(0, 0)$ and $(1, 1)$, each with probability $1/2$. Joint entropy is one bit: knowing the first label determines the second. If Y is instead an independent fair label, all four pairs have probability $1/4$, giving two bits of joint entropy. The marginal tables are the same in the two cases, while their dependence differs.

5.7 Coding blocks improves fractional averages

Return to three equally likely labels. A single-label prefix code with lengths $(1, 2, 2)$ averages $5/3$ bits per label. For a block of two labels there are nine equally likely ordered pairs. A binary tree can assign seven of these pairs length three and the remaining two length four. Its Kraft sum is $7/8 + 2/16 = 1$, and such a tree can be formed by splitting one leaf of a complete depth-three tree into two children.

The expected length per pair is $(7 \cdot 3 + 2 \cdot 4)/9 = 29/9$ bits, or $29/18 \approx 1.6111$ bits per original label. This is closer to $\log_2 3 \approx 1.5850$ than $5/3 \approx 1.6667$. The calculation exhibits how whole-number lengths for blocks allow smaller fractional averages per symbol.

Example 5.6 (An explicit two-label code)**.** List the nine ordered pairs as $AA, AB, AC, BA, BB, BC, CA, CB, CC$. Assign the first seven the words $000, 001, 010, 011, 100, 101, 110$, and assign the last two $1110, 1111$. No word prefixes another. Each pair is equally likely, so the expected length is $29/9$ bits per pair, as calculated above.

5.8 Interpreting a change of labels

Entropy depends on probabilities and is unchanged by merely renaming or permuting labels. Splitting or merging categories is a different operation. Splitting a label into two distinguishable sublabels can increase entropy because there is more to specify. Merging labels can remove that distinction. For example, merging the two probability-$1/4$ labels in $(1/2, 1/4, 1/4)$ gives $(1/2, 1/2)$ and reduces entropy from $3/2$ bits to one bit. Later, a stochastic channel will provide a general language for such recording operations.

5.9 Exercises

5.1 Evaluate $\log_2(1/16)$, $\ln(e^3)$, and $\log_2 32$.

5.2 Find the surprisal in bits of outcomes with probabilities $1/2$, $1/4$, and $1/32$.

5.3 Compute the entropy in bits of $(1/2, 1/8, 1/8, 1/4)$.

5.4 Explain why a certain outcome contributes zero entropy and why an impossible outcome is assigned zero contribution to the average.

5.5 Construct a prefix code with lengths $(1, 2, 3, 3)$. Find its expected length under the probabilities in Exercise 3, assigning the shortest code to the most probable label.

5.6 Can code lengths $(1, 1, 2)$ belong to a prefix code? Use the Kraft inequality.

5.7 For uniform probabilities on three labels, compute the expected length of the code $0, 10, 11$ and compare it with entropy.

5.8 For binary counts $(3, 3)$, compute the multiplicity, its base-two logarithm per observation, and the entropy of the empirical distribution.

5.9 Derive the finite multiplicity bound, carefully explaining why all ordered samples with the same counts have equal probability under q.

5.10 For fixed scores $(0, 1, 2)$, equal initial proportions, and $\eta = \ln 2$, find the proportions at step three.

5.11 Show that adding a common value to every fixed score leaves all log-odds increments unchanged.

5.12 Give two distributions on four labels with different entropy but the same support. Explain their different average surprisals.

5.10 Selected solutions

1. The values are -4, 3, and 5.

3. The entropy is $1/2 + 3/8 + 3/8 + 1/2 = 7/4$ bits.

5. One code is $0, 10, 110, 111$, assigned in decreasing probability order. The expected length is $(1/2)1 + (1/4)2 + (1/8)3 + (1/8)3 = 7/4$ bits.

6. No. The Kraft sum is $1/2 + 1/2 + 1/4 = 5/4 > 1$.

8. The multiplicity is $\binom{6}{3} = 20$, so the logarithmic count per observation is $\log_2 20/6 \approx 0.7203$ bits. The entropy is one bit.

10. The unnormalized weights are $(1, 8, 64)$, giving $(1/73, 8/73, 64/73)$.

Laboratory investigation

1. Calculate the surprisals and entropy of $(1/2, 1/4, 1/4)$ in both bits and nats. Show explicitly how division by $\ln 2$ converts between the units.
2. Draw the prefix tree for codewords $0, 10, 11$. Explain why a concatenation of these words can be decoded unambiguously. Compare its expected length under $(1/2, 1/4, 1/4)$ and under uniform probabilities.
3. In the demonstration, move probability mass toward a vertex, toward an edge midpoint, and toward the center. Record entropy and expected code length. Keep the code fixed and explain why its expected length need not equal entropy.
4. For binary counts $(2, 2)$, $(4, 4)$, and $(8, 8)$, compute the multiplicities and their base-two logarithms per observation. Compare them with one bit. Derive the exact finite upper bound rather than assuming equality.
5. For fixed scores $(0, 1, 2)$ and intensity $\ln 2$, start with equal probabilities and compute three updates through accumulated scores. Verify that every log-odds increment equals intensity times score difference.